

Streams



Definição

- Stream é uma sequência de eventos assíncronos. É definido um fluxo e quando adicionamos dados a esse fluxo ele será capaz de atualizar uma interface com o usuário.
- O uso de Streams nos direciona a uma programação mais reativa, ou seja, avisamos da mudança de dados e o Flutter promove a atualização dos widgets.
- Como primeiro exemplo veremos a classe **BotaoAzulServicoWeb** utilizado na aula anterior.



BotaoAzulServicoWeb

- Na definição do widget não foi implementado nada acerca do uso de Streams.
- O mais importante desse slide é a importação de **'dart:async'** para que possamos usar Streams.
- O resto do código basicamente define os atributos que serão customizados pelo programador.

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:somaweb/service/resposta_servico.dart';
import 'package:somaweb/widget/botao_azul.dart';

class BotaoAzulServicoWeb extends StatefulWidget {
  final String texto;
  final double? tamanho_fonte;
  final Color? cor_fonte;
  final bool Function()? pre_servico;
  final Future<RespostaServico> Function() acionar_servico;
  final Function(RespostaServico resposta)? pos_servico;
  final FocusNode? marcador_foco;

  BotaoAzulServicoWeb(
    {this.texto = "",
    this.tamanho_fonte,
    this.cor_fonte,
    this.pre_servico = null,
    required this.acionar_servico,
    this.pos_servico = null,
    this.marcador_foco});

  @override
  _BotaoAzulServicoWebState createState() => _BotaoAzulServicoWebState();
}
```



BotaoAzulServicoWeb

```
class _BotaoAzulServicoWebState extends State<BotaoAzulServicoWeb> {  
  final _streamController = StreamController<bool>();  
  
  @override  
  void initState() {  
    // TODO: implement initState  
    super.initState();  
  }  
  
  @override  
  void dispose() {  
    // TODO: implement dispose  
    super.dispose();  
    _streamController.close();  
  }  
}
```

- Aqui já temos algo muito importante; a definição do **StreamController**. Esse objeto ficará responsável por controlar o fluxo da Stream. É ele que será avisado da mudança de um dado.
- Notar que o **StreamController** recebe o tipo de dado que ele irá controlar, nesse caso um **bool**.
- Uma outra questão importante é a chamada do método **close()** quando o fluxo da Stream não for mais necessário.



build

- O widget **StreamBuilder** recebe o tipo de dado que ele irá tratar (deve ser o mesmo que o **StreamController**).
- Em **stream** setamos a *stream* do StreamController definido no widget (**_streamController**).
- **initialData** recebe o valor inicial (false) que será passado em **snapshot.data** para **mostrar_progress**.
- Quando **_streamController.add(true)** é acionado o **StreamBuilder** reconstrói o BotaoAzul e **snapshot.data** terá o valor **true** (fazendo o progress aparecer).
- Quando **_streamController.add(false)** é acionado o **StreamBuilder** reconstrói o BotaoAzul e **snapshot.data** terá o valor **false** (fazendo o progress desaparecer).

```
@override
Widget build(BuildContext context) {
  return StreamBuilder<bool>(
    stream: _streamController.stream,
    initialData: false,
    builder: (context, snapshot){
      return BotaoAzul(
        texto: widget.texto,
        mostrar_progress: snapshot.data!,
        ao_clicar: () async {
          if (widget.pre_servico == null) return null;
          else {
            // Se pre_servico retorna false simplesmente retorna (não aciona
            // o serviço)
            if (!(widget.pre_servico!())){
              return;
            }
          }
        }
      );
      // Vai colocar o progress visível
      _streamController.add(true);

      // Aciona o serviço
      RespostaServico resposta = await widget.acionar_servico();

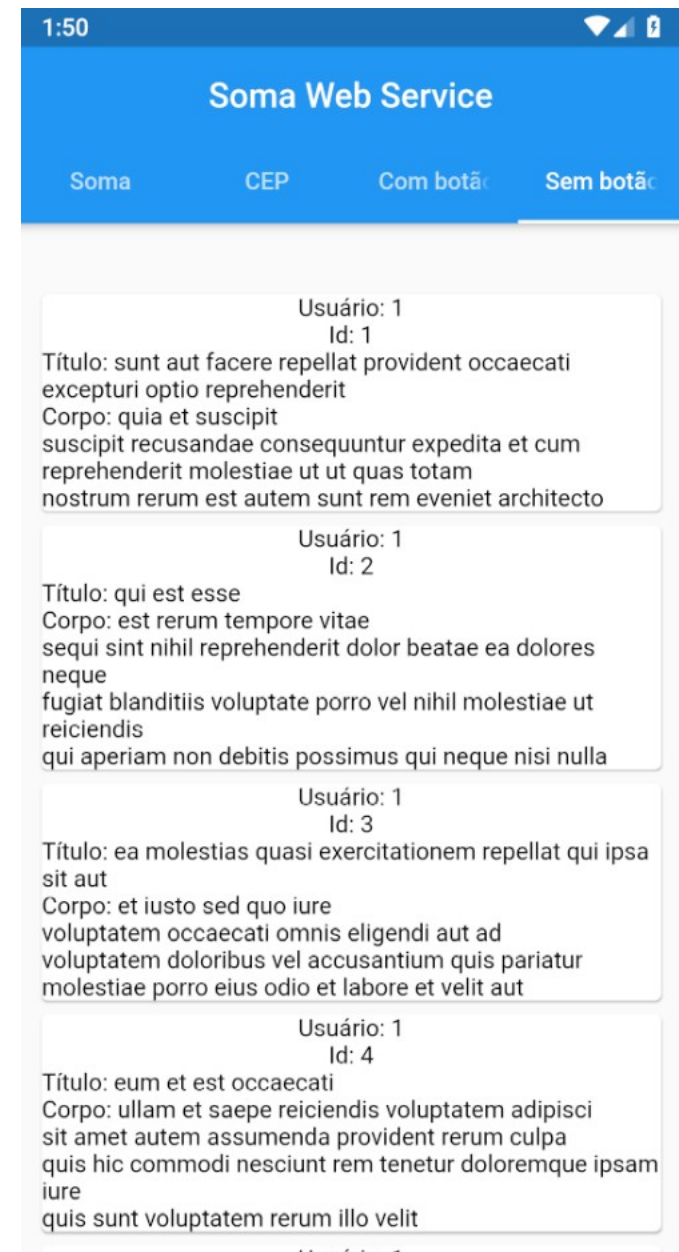
      // Vai voltar com o texto para o botão
      _streamController.add(false);

      // Vai atualizar a tela com os resultados obtidos no serviço
      widget.pos_servico!(resposta);
    },
  ); // BotaoAzul
},
); // StreamBuilder
}
```



Segundo exemplo

- Também utilizamos Streams no carregamento dos dados da Tab_Posts2 (Sem botão).
- O ListView utilizado (ListViewPostCard) é o mesmo da última aula, mas para obter a listagem para sua exibição utilizamos Streams e a separação entre o controle de interação (parte não visual) e o desenho dos widgets (parte visual). Esse é o mesmo princípio utilizado no padrão BLoC (Business Logic of Component). Com o padrão BLoC é possível separar lógica de negócio da interface.
- O link: <https://flutterparainiciantes.com.br/gerenciamento-estado/bloc/> possui maiores orientações acerca do padrão BLoC.



ControleStreamTabPosts2

```
import 'dart:async';

class ControleStreamTabPosts2{
  final _controlador = StreamController<RespostaServico>();
  Stream<RespostaServico> get fluxo => _controlador.stream;

  Future<RespostaServico<List<Post>>> obterPosts() async{

    // Obtendo os posts
    RespostaServico<List<Post>> resposta = await PostsService.obterPosts();

    // Atualizando o fluxo dos dados (vai fazer o StreamBuilder relacionado
    // a esse _controlador atualizar a visualização dos dados)
    _controlador.add(resposta);

    return resposta;
  }

  void dispose() {
    _controlador.close();
  }
}
```

- **ControleStreamTabPosts2** possui internamente um **StreamController** que deve ser tipado de acordo com o dado que o seu respectivo **StreamBuilder** irá trabalhar, nesse caso **RespostaServico**.
- No método **obterPosts** temos a chamada ao serviço de obtenção de Posts e atualização do fluxo através da chamada ao método **add**.
- Na prática essa chamada ao método **add** irá acionar o **StreamBuilder** que estiver ligado ao **StreamController** do nosso **ControleStreamTabPosts2**. E isso irá atualizar a interface gráfica (o widget que se encontra dentro de StreamBuilder, no nosso caso o ListViewPostCard).



TabPosts2

- Essa parte inicial do código não trata do uso de Streams.
- Tem-se aqui a questão dessa ser uma Tab Persistente, ou seja, que é necessário fazer o uso do mixin **AutomaticKeepAliveClientMixin** e a definição de **wantKeepAlive** para **true**.
- Notar que não há a necessidade de importar a biblioteca `import 'dart:async';` pois ela já está definida no **ControleStreamTabPosts2**.

```
import 'package:flutter/material.dart';
import 'package:somaweb service/domain/post_bloc.dart';
import 'package:somaweb service/services/api_response.dart';
import 'package:somaweb service/tabs/localwidget/listview_post_card.dart';

class TabPosts2 extends StatefulWidget {
  @override
  _TabPostsState2 createState() => _TabPostsState2();
}

// Esse "with AutomaticKeepAliveClientMixin<TabPosts2>" vai ser usado para não recarregar
// os dados dessa aba
class _TabPostsState2 extends State<TabPosts2> with AutomaticKeepAliveClientMixin<TabPosts2>{
  // Mantém a aba ativa para não recarregamento dos dados
  @override
  // TODO: implement wantKeepAlive
  bool get wantKeepAlive => true;
}
```



TabPosts2

```
// Controlador não visual para atuar com StreamBuilder
final _controleStreamBuilder = ControleStreamTabPosts2();

@override
void initState() {
  // TODO: implement initState
  super.initState();
  _controleStreamBuilder.obterPosts();
}

@override
void dispose() {
  // TODO: implement dispose
  super.dispose();
  _controleStreamBuilder.dispose();
}
```

- Aqui temos a definição do **ControleStreamTabPosts2** (que irá encapsular o **StreamController**).
- No **initState()** é feita a chamada para a obtenção dos posts. Como essa chamada é assíncrona, o método **build** (que será apresentado nos próximos slides) será acionado antes mesmo de seu término. Entretanto, quando o serviço terminar o **StreamController** (dentro do **_controleStreamBuilder**) irá acionar a reconstrução da tab.
- Já no **dispose()** temos a liberação do fluxo de dados do **StreamController** que se encontra dentro do **_controleStreamBuilder**.



build

```
@override
Widget build(BuildContext context) {
  // Não pode esquecer dessa linha para não acontecer o recarregamento
  super.build(context);

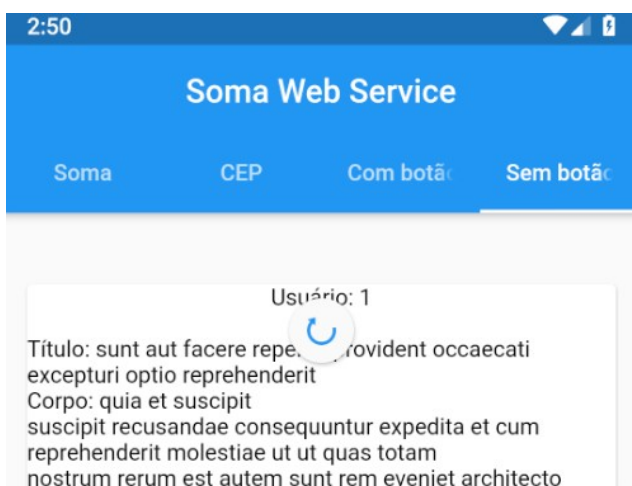
  return StreamBuilder<RespostaServico>(
    stream: _controleStreamBuilder.fluxo,
    builder: (context, snapshot) {
      // Vai colocar um progress enquanto os posts não forem carregados
      if (!snapshot.hasData) {
        return Center(
          child: CircularProgressIndicator(),
        ); // Center
      }

      RespostaServico resposta = snapshot.data!;

      // Se houve algum erro
      if (!(resposta.sucesso))
        return Center(
          child: Text(
            resposta.mensagemErro!,
            style: TextStyle(
              color: Colors.red,
              fontSize: 22,
            ), // TextStyle
          ), // Text
        ); // Center
    }
  );
}
```

- Aqui temos a primeira parte do método **build**.
- No parâmetro **stream**, da definição do **StreamBuilder**, é setado o **fluxo** do **_controleStreamBuilder** (que é o stream do StreamController dentro de ControleStreamTabPosts2).
- No método definido para o parâmetro **builder** temos o uso de **snapshot.hasData**. Isso é utilizado para sabermos se já temos dados para serem apresentados, se não tivermos é exibido um Progress.
- Se já temos dados, não cairemos no **if(!snapshot.hasData)** e obteremos esses dados, que podem ser uma falha (o serviço recebeu uma mensagem de erro, por exemplo) ou a lista de Posts. Ao lado temos o código que trata o caso de uma falha.





```
// RefreshIndicator é um widget para atualizar a lista de posts
// deslizando o dedo para baixo
return RefreshIndicator(
  onRefresh: _controleStreamBuilder.obterPosts,
  // Aqui temos a lista de posts sendo informada para o ListView
  child: ListViewPostCard(resposta.resultado)
); // RefreshIndicator
},
); // StreamBuilder
}
```

RefreshIndicator

- Aqui poderíamos ter apenas o **return ListViewPostCard(resposta.resultado)**. Mas optamos por usar um **RefreshIndicator** para atualizar a listagem caso o usuário “puxe” a listagem para baixo.
- **onRefresh** recebe um método que retorna um **Future** e quando esse Future estiver completo, o **RefreshIndicator** retira o progress e atualiza seu **child**.
- A listagem então vem pronta em **resposta.resultado** (lembrar que **resposta** veio de **snapshot.data** – ver slide anterior).



Dúvidas?

